

# Trees

DSA27 — Lecture 11 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Faculty of Computers and Information Sciences, Mansoura University

Week 11 · Fall 2026

## Where We Are

---

## From lines to branches

Every structure so far was **linear**: array, linked list, stack, queue.

- A sorted array: search  $O(\log n)$ , but insert  $O(n)$  — everything shifts.
- A linked list: insert  $O(1)$  once you are there, but search  $O(n)$ .

Today: a structure where search **and** insert cost  $O(\log n)$  — **while it stays balanced**.

# Today

---

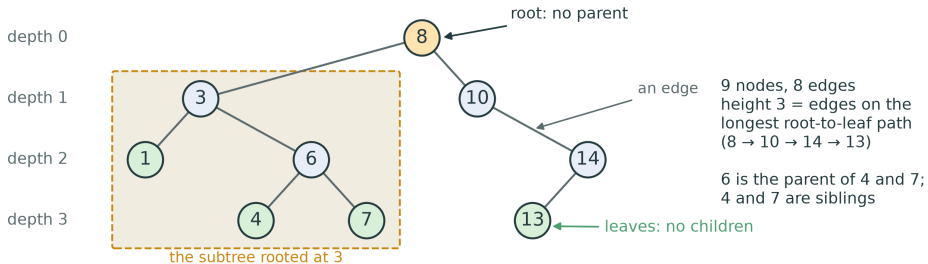
1. Vocabulary: root, leaf, depth, height, subtree
2. Binary trees, and the TreeNode
3. The BST property — and why checking parents is not enough
4. Search, insert, min and max: one path down
5. Delete: three cases, and the in-order successor
6. The four traversals — and what each one is **for**
7. When the tree is a linked list: measured

# The Vocabulary

---

# A tree, and its words

**A tree: one root, every other node has exactly one parent**



# Depth and height

- **Depth** of a node: edges from the **root** down to it. The root has depth 0.
- **Height** of a node: edges on the longest path from it **down** to a leaf. A leaf has height 0.
- **Height of the tree** = height of the root. Empty tree:  $-1$ .

|                        | depth — العمق  | height — الارتفاع      |
|------------------------|----------------|------------------------|
| measured from the root | the root, down | the leaves, up         |
| a leaf                 | 0<br>its level | the tree's height<br>0 |

# Trees are everywhere

- your **file system**: folders inside folders
- a web page: the HTML **DOM**
- an **expression** —  $(3 + 4) * 2$  — as your compiler sees it
- a **decision** tree; a tournament bracket; a family tree
- the **recursion tree** of `fib(n)` (Lecture 03)



# Binary Trees

---

## At most two children

A **binary tree**: every node has at most two children, a **left** and a **right**. In Arabic: شجرة ثنائية.

```
class TreeNode:
    __slots__ = ("value", "left", "right")

    def __init__(self, value, left=None, right=None):
        self.value = value
        self.left = left
        self.right = right
```

Lecture 05's Node had one link, next. This one has two.

## How tall must a tree be?

- Level  $d$  holds **at most**  $2^d$  nodes: 1, 2, 4, 8, ...
- A tree of height  $h$  holds **at most**  $2^{h+1} - 1$  nodes.
- So  $n$  nodes need height **at least**  $\lfloor \log_2 n \rfloor$  — and may have height up to  $n - 1$ .

| $n$       | shortest possible height | tallest possible height |
|-----------|--------------------------|-------------------------|
| 7         | 2                        | 6                       |
| 1,000     | 9                        | 999                     |
| 1,000,000 | 19                       | 999,999                 |

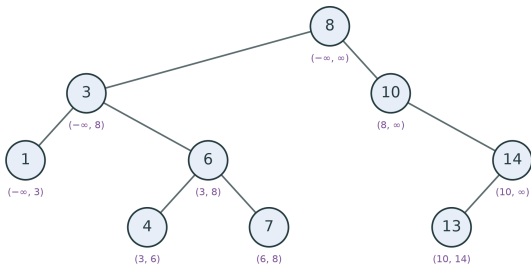
# Binary Search Trees

---

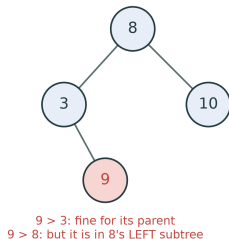
# The BST property

## The BST property: left subtree smaller, right subtree larger — all of it

valid: each value lies inside the range its ancestors allow



not a BST, though every parent-child pair looks right



For **every** node  $x$ : everything in  $x$ 's **left subtree** is smaller than  $x$ , and everything in its **right subtree** is larger.

## is\_valid: carry the range down

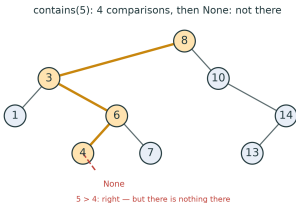
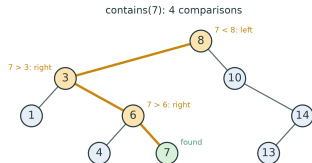
The wrong check, at each node: `left.value < node.value < right.value`.

The right check: pass an allowed range (**low, high**) down the recursion.

- going **left** from x: the new high is x
- going **right** from x: the new low is x
- the empty tree is valid

# Search: one path down

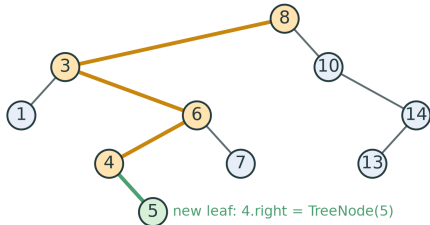
Search goes down one path: at most  $h + 1$  comparisons



```
node = self.root
while node is not None:
    if value == node.value:
        return True
    node = node.left if value < node.value else node.right
return False
```

# Insert: a search that fails

Insert = a search that fails, then one new leaf where it stopped



```
insert(5):  
5 < 8: left  
5 > 3: right  
5 < 6: left  
5 > 4: right - None,  
so the new node goes here
```

1. Empty tree → the new node **is** the root.
2. Otherwise walk down as in search. Equal → already there: do nothing.
3. When the way on is None, hang the new node **there**.

A new value always becomes a **leaf**. Nothing moves:  $O(h)$ .



## Min and max: no comparisons at all

- **min**: from the root, go **left** until there is no left child.
- **max**: from the root, go **right** until there is no right child.
- Empty tree  $\rightarrow$  `ValueError`.

$O(h)$ , and not a single comparison of values.

## **Delete: Three Cases**

---

## Find it, then look at its children

Find the node, **remembering its parent**. Then:

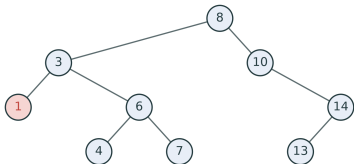
| The node has...              | Do this                                                           |
|------------------------------|-------------------------------------------------------------------|
| no children (a <b>leaf</b> ) | the parent's link becomes None                                    |
| <b>one</b> child             | the parent's link points at that child                            |
| <b>two</b> children          | copy in the <b>in-order successor</b> , then delete the successor |

Deleting a value that is not there does nothing.

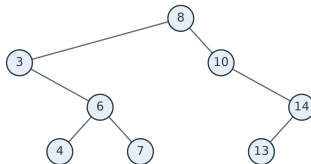
# The easy cases

The easy cases: a leaf is cut off; a single child takes its parent's place

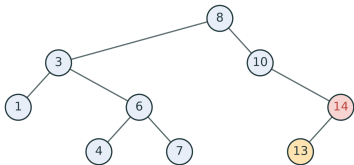
leaf: delete(1)



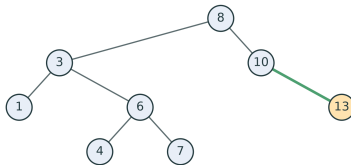
after: 3.left = None



one child: delete(14) — its only child is 13



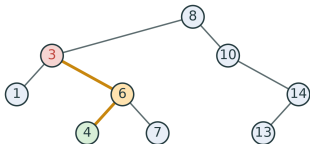
after: 10.right = 13 — the child moves up



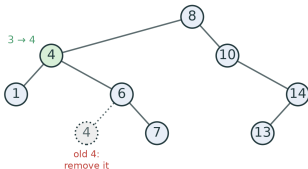
# Two children: the in-order successor

Two children: replace the value with the in-order successor, then delete the successor

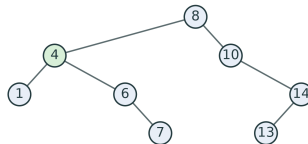
1. find the successor: right once, then left until None → 4



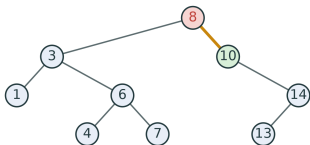
2. copy 4 into the node that held 3 (two 4s for a moment)



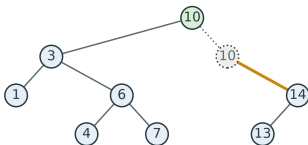
3. delete the old 4 — a leaf. Still valid:  $1 < 4 < 6$  and 7



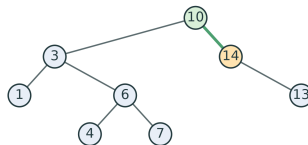
1. delete(8): right once → 10; 10.left is None, so 10 is it



2. copy 10 into the root; old 10 has ONE child, 14



3. splice 14 into the old 10's place (the one-child case)



## The three cases collapse into two

1. Find the node and its parent. Not found → return.
2. **Two children?** Walk to the successor (keeping *its* parent), copy its value up — and from now on, delete **the successor node** instead.
3. Now the node has **at most one** child, child (possibly None):
  - no parent → `self.root = child`
  - node was its parent's left → `parent.left = child`
  - otherwise → `parent.right = child`

# The Four Traversals

---

## Recursion on trees

A **traversal** visits **every** node once — in Arabic, اجتياز الشجرة. For a binary tree, every recursive walk has the same skeleton:

- **base case:** the empty tree — do nothing
- **recursive case:** do something with the node, and walk the left and right subtrees

*Where* you put “do something” gives three different orders.



## In-order: the code

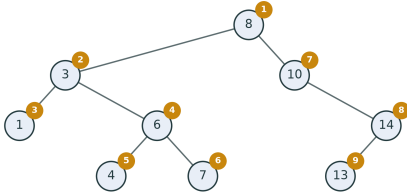
```
def in_order(self):  
    out = []  
    self._in_order(self.root, out)  
    return out  
  
def _in_order(self, node, out):  
    if node is not None:  
        self._in_order(node.left, out)  
        out.append(node.value)  
        self._in_order(node.right, out)
```

Move `out.append` **up** one line → pre-order. **Down** one line → post-order.

# Four walks of one tree

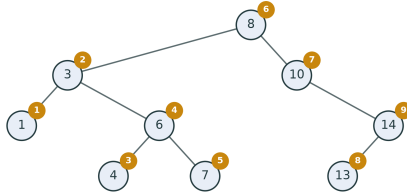
## Four walks of the same tree — the amber numbers are the visit order

pre-order: node, left, right



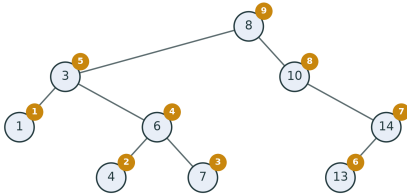
8, 3, 1, 6, 4, 7, 10, 14, 13

in-order: left, node, right



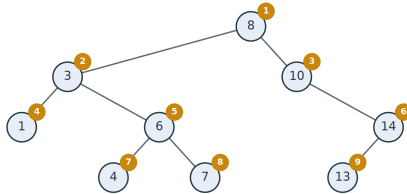
1, 3, 4, 6, 7, 8, 10, 13, 14

post-order: left, right, node



1, 4, 7, 6, 3, 13, 14, 10, 8

level-order: row by row, left to right



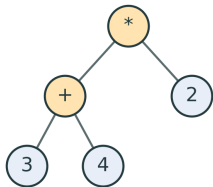
8, 3, 10, 1, 6, 14, 4, 7, 13

## What each one is FOR

| Walk               | Gives you                    | Used for                                                                       |
|--------------------|------------------------------|--------------------------------------------------------------------------------|
| <b>in-order</b>    | a BST's values <b>sorted</b> | listing, range queries, <code>is_valid</code>                                  |
| <b>pre-order</b>   | parents before children      | <b>copying</b> / saving a tree: re-inserting it <b>rebuilds</b> the same shape |
| <b>post-order</b>  | children before parents      | <b>freeing</b> a tree; <b>evaluating</b> an expression; sizes and heights      |
| <b>level-order</b> | nearest to the root first    | printing by level; shortest distances (BFS, Week 14)                           |

# Post-order evaluates: expression trees

**An expression tree for  $(3 + 4) * 2$ : the shape holds the precedence**



pre-order     $* + 3 4 2$     prefix (Polish) notation

in-order     $3 + 4 * 2$     infix — but the brackets are lost!

post-order     $3 4 + 2 *$     postfix: Lecture 06's stack evaluates it  $\rightarrow 14$

# Level-order needs a queue

**level\_order(): the queue holds the nodes found but not yet visited**

| step | children added | queue after the step (front on the left) |
|------|----------------|------------------------------------------|
|------|----------------|------------------------------------------|

|            |               |             |
|------------|---------------|-------------|
| start      |               | 8           |
| dequeue 8  | enqueue 3, 10 | 3 10        |
| dequeue 3  | enqueue 1, 6  | 10 1 6      |
| dequeue 10 | enqueue 14    | 1 6 14      |
| dequeue 1  | –             | 6 14        |
| dequeue 6  | enqueue 4, 7  | 14 4 7      |
| dequeue 14 | enqueue 13    | 4 7 13      |
| dequeue 4  | –             | 7 13        |
| dequeue 7  | –             | 13          |
| dequeue 13 | –             | empty: stop |

visited: 8, 3, 10, 1, 6, 14, 4, 7, 13

1. Enqueue the root.
2. While the queue is not empty: **dequeue** a node, visit it, **enqueue** its children (left, then right).

## Height and size: post-order in one line

```
def _height(self, node):  
    if node is None:  
        return -1  
    return 1 + max(self._height(node.left), self._height(node.right))
```

size is the same shape: 0 for None, else 1 + left size + right size.

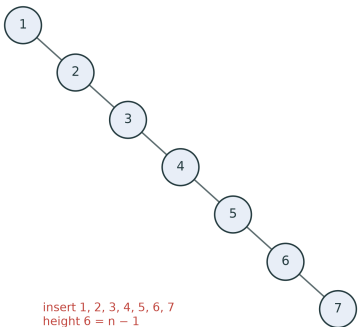
## When the Tree Is a Linked List

---

# Same values, same code, different height

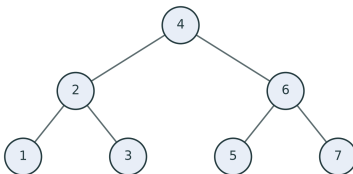
## Same seven values, same code — the insertion order decides the height

sorted order: a linked list in a tree costume



insert 1, 2, 3, 4, 5, 6, 7  
height 6 =  $n - 1$   
contains(7): 7 comparisons

middle first: as short as 7 nodes can be

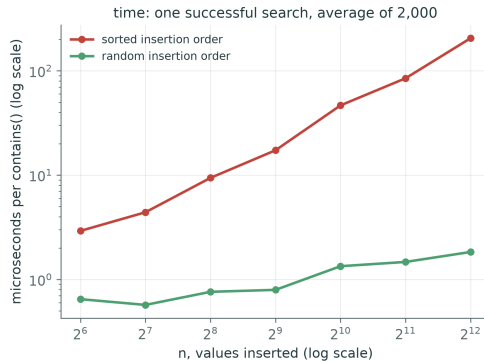
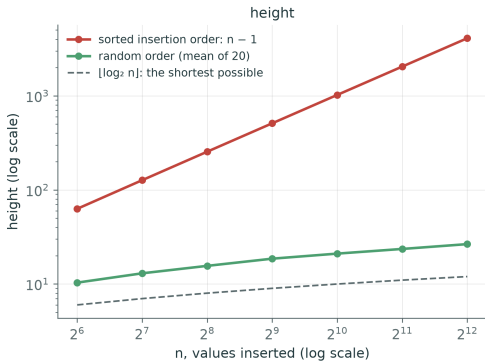


insert 4, 2, 6, 1, 3, 5, 7  
height 2 =  $\lceil \log_2 7 \rceil$   
contains(7): 3 comparisons

Insert **sorted** values and each new one is the largest so far: it always goes **right**.  
Height  $n - 1$ . Every operation:  $O(n)$ .



## Measured: a BST is only as fast as it is short



## The cure: keep it balanced — beyond this course

**Balanced** trees re-shape themselves on every insert and delete, so that  $h$  stays  $O(\log n)$  whatever the input order:

- **AVL trees** (Adelson-Velsky and Landis, 1962)
- **red-black trees** (Guibas and Sedgwick, 1978)

Both are **beyond the bylaw**: named here, not examined.

## What each operation costs

| Operation                    | Cost   | Short tree   | Chain  |
|------------------------------|--------|--------------|--------|
| contains, insert, delete     | $O(h)$ | $O(\log n)$  | $O(n)$ |
| min, max                     | $O(h)$ | $O(\log n)$  | $O(n)$ |
| height, size, is_valid       | $O(n)$ |              |        |
| the four traversals          | $O(n)$ |              |        |
| extra space, recursive walks | $O(h)$ | $O(\log n)$  | $O(n)$ |
| extra space, level_order     | $O(w)$ | up to $O(n)$ | $O(1)$ |

## **This Week**

---

## Exercises: dsa/tree.py

| Method              | Target | The trap                                                            |
|---------------------|--------|---------------------------------------------------------------------|
| insert              | $O(h)$ | stop <b>at the parent</b> ;<br>ignore duplicates                    |
| contains, min, max  | $O(h)$ | a loop; ValueError on empty                                         |
| delete              | $O(h)$ | keep the parent; the <b>root</b> ;<br>successor = right child       |
| height, size        | $O(n)$ | empty tree is $-1$ and $0$                                          |
| is_valid            | $O(n)$ | carry <b>(low, high)</b> down,<br>not just the parent               |
| the four traversals | $O(n)$ | return a <b>new list</b> ;<br>level-order uses <b>your</b><br>queue |

```
pytest tests/test_tree.py -v
```

## Homework 11 — before Lecture 12

1. **Implement** `dsa/tree.py` until all 26 tests pass.
2. **Draw** the BST built by inserting 50, 30, 70, 20, 40, 60, 80, 35, 45, 65, then delete 20, 30 and 50 — the tree after each delete.
3. **Walk** your tree of item 2 (before the deletes) in all four orders, by hand. Check with your code.
4. **Measure** the height of a BST built from `range(n)` and from a shuffled `range(n)`, for `n` up to 900, in `notebooks/11-trees.ipynb`. Why stop at 900?

## Summary

---

## Seven things to keep

1. A tree: one root; every other node has **one parent**;  $n$  nodes,  $n - 1$  edges.
2. **Height** counts edges down to the deepest leaf; empty tree  $-1$ , leaf  $0$ .
3. BST: **left subtree smaller, right subtree larger** — for the whole subtree.
4. Search, insert, min, max: **one path down**,  $O(h)$ .
5. Delete with two children: copy the **in-order successor**, delete it instead.
6. In-order is **sorted**; pre-order **rebuilds**; post-order **frees and evaluates**; level-order needs a **queue**.
7. Sorted input gives height  $n - 1$ :  $O(n)$ . Balanced trees fix it — beyond this course.



**Week 12 — Heaps and priority queues.** A tree with no links at all: a **complete** binary tree stored in an Array, where the children of slot  $i$  are slots  $2i + 1$  and  $2i + 2$ .